

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «Аппаратно-программное обеспечение ЭВМ и сетей»
Тема: «Изучение аппаратно-программной
архитектуры процессоров семейства Intel – разработка
ассемблерной вставки»

Выполнила:
Студент 4-го курса
Группы АС-66
Демидович А.А.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: изучение программной модели микропроцессора, регистров, организации памяти; практическая работа с ассемблерными вставками

Ход работы

Задания:

1. Изучить следующие теоретические сведения: программная модель микропроцессора; регистры и их назначение; организация памяти; основные конструкции и базовые команды ассемблера (используя материалы лекций и методические указания).
2. Написать ассемблерную вставку, реализующую обработку строки согласно варианту, выданному преподавателем. Оформить ее в виде отдельной функции.
3. Реализовать данную обработку строки также в виде функции на C++. Сравнить быстродействие обоих вариантов. В отчете отразить выводы. Содержание отчета:

- титульный лист;
- описание варианта задания;
- текст программы;
- скриншоты результатов.

Индивидуальное задание:

4. Даны 2 строки. Совместить четные символы одной строки с нечетными другой.

Код:

```
#include <iostream>
#include <string>
#include <chrono>
#include <vector>
#include <cstring>

// 1. Реализация на чистом C++
void combineCpp(const char* s1, const char* s2, char* dest, int len) {
    for (int i = 0; i < len; ++i) {
        if (i % 2 == 0) {
            dest[i] = s1[i]; // 1-й, 3-й, 5-й символы (индексы 0, 2, 4...) из первой
            строки
        }
        else {
            dest[i] = s2[i]; // 2-й, 4-й, 6-й символы (индексы 1, 3, 5...) из второй
            строки
        }
    }
    dest[len] = '\0';
}

// 2. Реализация со встроенной ассемблерной вставкой (__asm)
void combineAsm(const char* s1, const char* s2, char* dest, int len) {
    __asm {
        mov esi, s1; esi = указатель на s1
        mov ebx, s2; ebx = указатель на s2
        mov edi, dest; edi = указатель на результирующую строку
        mov ecx, len; ecx = количество символов
        xor edx, edx; edx = индекс i = 0
```

```

START_LOOP:
cmp edx, ecx
    jge END_LOOP; если i >= len, завершить цикл

    test edx, 1; проверяем младший бит индекса(четность)
    jnz TAKE_FROM_S2; если не 0 (нечетный индекс 1, 3, 5...), берем из s2

    TAKE_FROM_S1 :
    mov al, [esi + edx]; берем байт из s1[i]
    jmp STORE_CHAR

    TAKE_FROM_S2 :
    mov al, [ebx + edx]; берем байт из s2[i]

    STORE_CHAR:
    mov[edi + edx], al; записываем в dest[i]
    inc edx; i++
    jmp START_LOOP

    END_LOOP :
    mov byte ptr[edi + edx], 0; ставим нулевой терминатор '\0'
}
}

int main() {
    setlocale(LC_ALL, "Russian");

    // Демонстрация корректности работы
    const char* str1 = "1111111111";
    const char* str2 = "2222222222";
    int len = strlen(str1);

    char resCpp[100];
    char resAsm[100];

    combineCpp(str1, str2, resCpp, len);
    combineAsm(str1, str2, resAsm, len);

    std::cout << "--- Проверка работы функций ---\n";
    std::cout << "Строка 1: " << str1 << "\n";
    std::cout << "Строка 2: " << str2 << "\n";
    std::cout << "Результат C++: " << resCpp << "\n";
    std::cout << "Результат Asm: " << resAsm << "\n\n";

    // Тестирование быстродействия на большой строке
    const int TEST_LEN = 10000000; // 10 млн символов
    std::string bigStr1(TEST_LEN, 'A');
    std::string bigStr2(TEST_LEN, 'B');
    std::vector<char> bigDestCpp(TEST_LEN + 1);
    std::vector<char> bigDestAsm(TEST_LEN + 1);

    // Замер C++
    auto startCpp = std::chrono::high_resolution_clock::now();
    combineCpp(bigStr1.c_str(), bigStr2.c_str(), bigDestCpp.data(), TEST_LEN);
    auto endCpp = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> elapsedCpp = endCpp - startCpp;

    // Замер ASM
    auto startAsm = std::chrono::high_resolution_clock::now();
    combineAsm(bigStr1.c_str(), bigStr2.c_str(), bigDestAsm.data(), TEST_LEN);
    auto endAsm = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> elapsedAsm = endAsm - startAsm;

    std::cout << "--- Сравнение быстродействия (" << TEST_LEN << " символов) ---\n";
    std::cout << "Время работы C++: " << elapsedCpp.count() << " мс\n";
    std::cout << "Время работы Asm: " << elapsedAsm.count() << " мс\n";

    return 0;
}

```

```
--- Проверка работы функций ---  
Строка 1: 111111111  
Строка 2: 222222222  
Результат C++: 1212121212  
Результат Asm: 1212121212  
  
--- Сравнение быстродействия (10000000 символов)  
Время работы C++: 4.5988 мс  
Время работы Asm: 4.229 мс
```